

Advanced Digital Image Processing Lecture #12

Dr. Md. Hasanuzzaman

Assistant Professor

Department of Computer Science & Engineering
Dhaka University

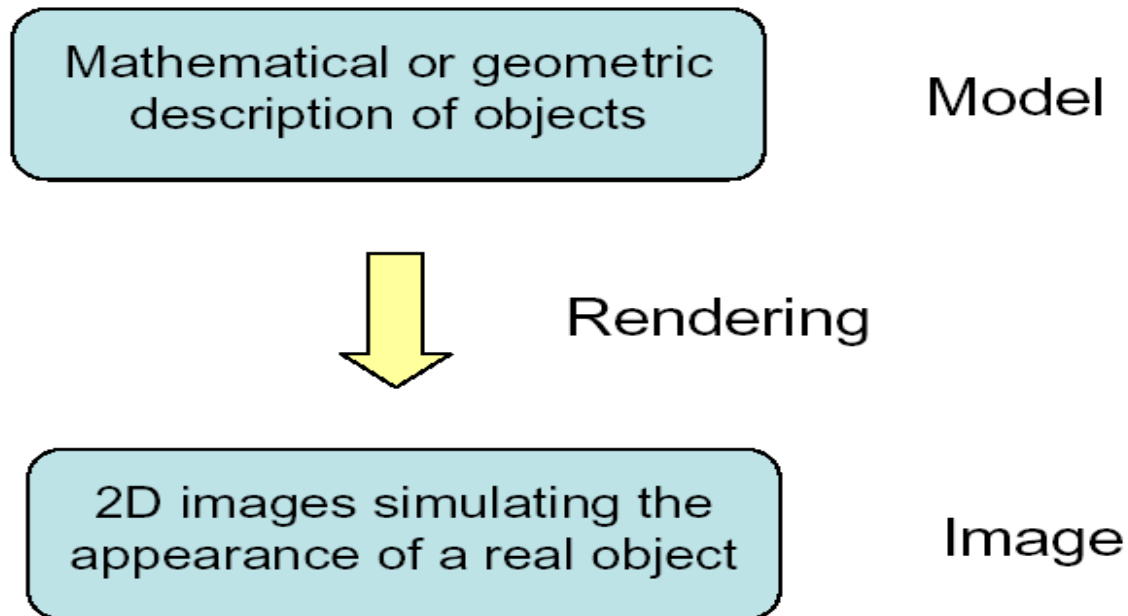
March 10, 2007

Contents

- ❖ What is Computer Graphics?
- ❖ Rendering?
- ❖ Rendering Techniques
 - ❑ Pipeline-Based Rendering
 - ❑ Ray-Tracking

Graphics & Rendering

What is Computer Graphics?



Another View

Rendering

+

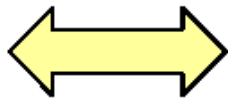
Modeling

+

Animation

+

Postprocessing

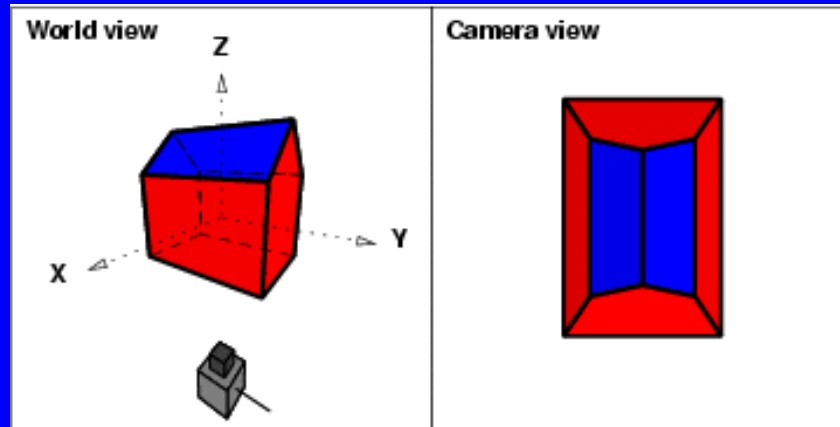


Computer Graphics

Rendering

- 3d to 2d projections, known as "rendering"
 - World and camera views
 - Mapping 3D data to 2D
 - Rotating objects to acquire a better viewing angle
 - Sorting faces and using Z buffers
 - More transformations

World and Camera Views



- In a 3D world, 3 axes denoting the X, Y and Z axes of three dimensional world,
- and a very basic interpretation of what a camera looks like. The grayish cube is the body of the camera, the black cube on top of it is the camera's lens, and the line coming out of the camera's bottom is supposed to depict a leg at the bottom of the camera, so we can understand what's up and down from the camera's viewpoint.
- In the right panel of the picture you can see what the camera sees, it's a camera view of our world,

Mapping 3D data to 2D

- The main goal of a renderer is to transform and project the 3D coordinates of the objects to the 2D screen.
- Assumes:
 - the camera at a fixed position.
 - the camera always looks up the Z direction, and always looks perpendicular to the XY plane.
 - The camera is placed in such a position that it sees the axes on the XY plane similar to the X and Y axes on a computer screen.
 - the X direction on a computer screen goes from left to right, where the Y direction goes from the top of your screen to the bottom of your screen.

Mapping 3D data to 2D

- As we're looking along the Z axis, all objects that should be rendered on our screen appear smaller as their coordinates have bigger Z values.
- Similarly, objects appear bigger as their Z values are smaller, since their coordinates are closer to the camera.
- In order to transform 3D coordinates to 2D coordinates, we can thus simply divide the X and Y values in the coordinates by the Z value in order to display them as X and Y coordinates in a 2 dimensional view.

$$\blacksquare x_{2d} = (x_{3d} / z_{3d}) ;$$

$$\blacksquare y_{2d} = (y_{3d} / z_{3d}) ;$$

Mapping 3D data to 2D

- Some Z values will be negative, and dividing a X , Y coordinate by this negative Z value will not make it bigger, but it will reverse the sign of our X and Y coordinates and we would be scaling these coordinates the wrong way.
- In order to overcome this problem, we add a number to the Z value so all Z values are strictly positive before we divide the X and Y values by this Z . This extra number is called a distance.

Mapping 3D data to 2D

- After computed the 2 dimensional X and Y values of the projected coordinates, it needs to scale the new coordinates up or down to make sure that the objects fit your viewport.
 - To use a scaling factor of your 2D coordinates so it fits your screen
- 2D image would be appear around the top left corner of the computer screen, since the origin of a computer screen sits at the top left of the screen
 - To add an X and Y offset to your computed 2D coordinates in order to put the projected objects in the middle of the screen
- The resulting code for projecting a 3D coordinate to a 2D coordinate using a camera following those assumption will look like,
 - $x_{2d} = x_{Offset} + scale * x_{3d} / (z_{3d} + distance);$
 $y_{2d} = y_{Offset} + scale * y_{3d} / (z_{3d} + distance);$

Basics of Rendering

- Pipeline Based Rendering
 - Objects in the scene are rendered in a sequence of steps that form the Rendering Pipeline.
- Ray-Tracing
 - A series of rays are projected thru the view plane and the view plane is colored based on the object that the ray strikes

Next Assignment (G4)

- Raster Scan Display Systems
 - Simple Raster Display System
 - Raster Display System With Peripheral Processor
 - Raster Display System with Integrated Display Processor

Next Assignment (G5)

- Color Perception
- Chromaticity diagram
- Color tone adjustment
- Neural Network for Color Segmentation

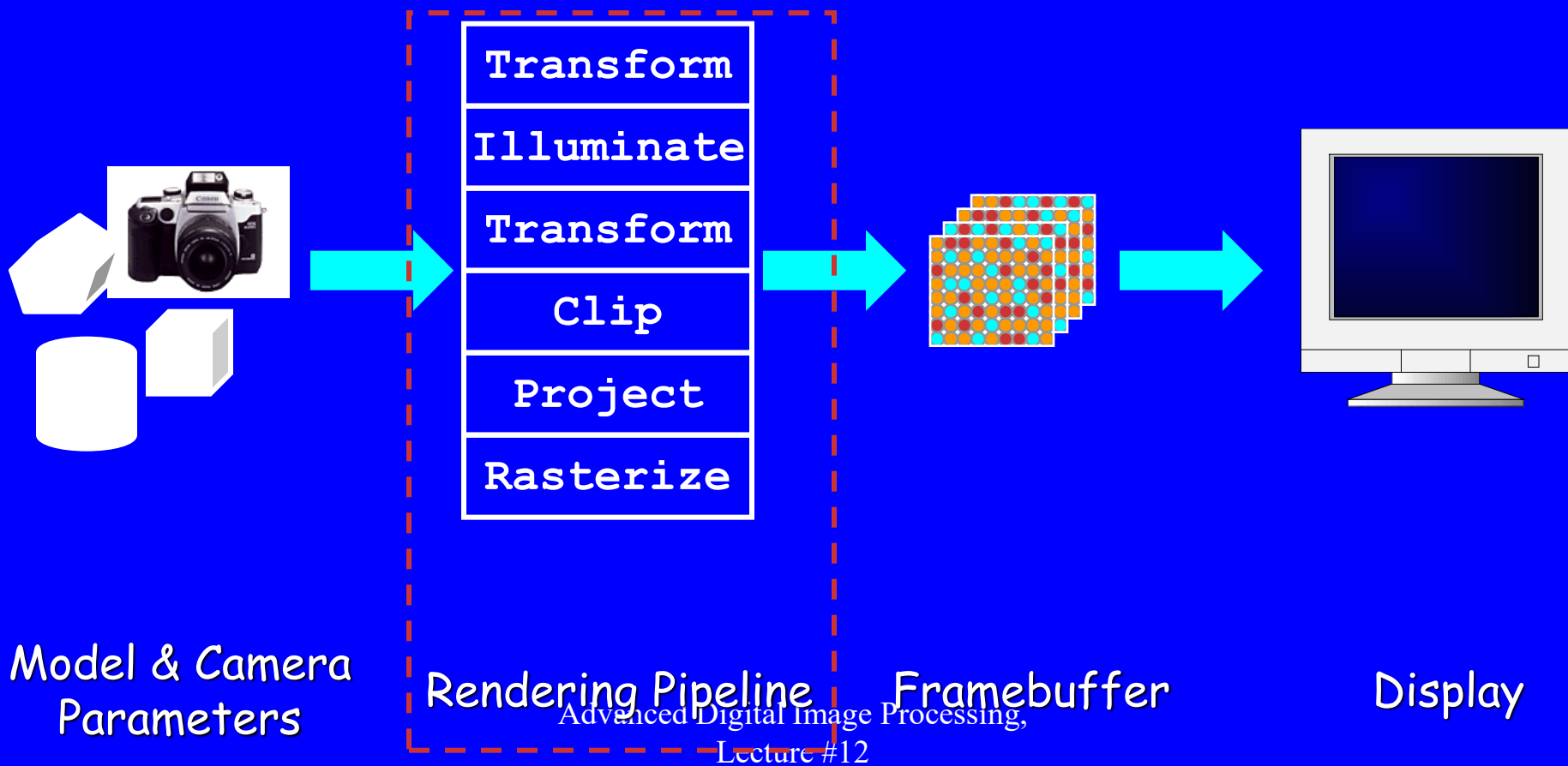
More About Rendering

More About Rendering

Scene Definitions



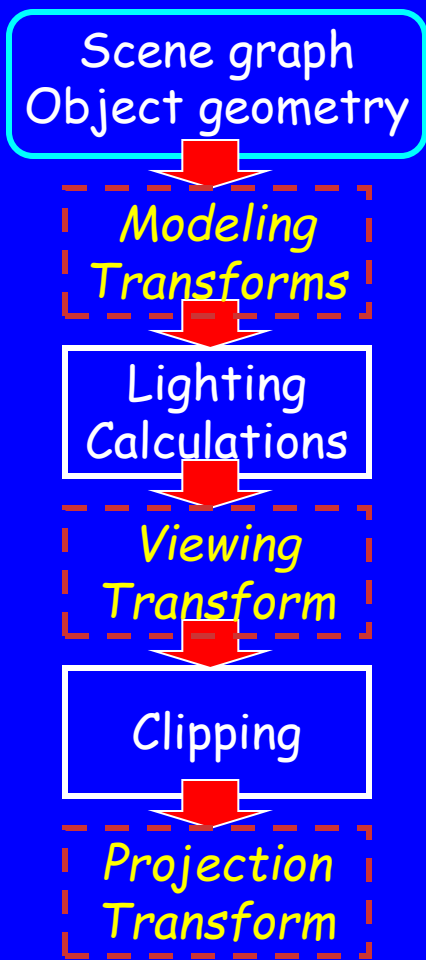
Pipeline Rendering



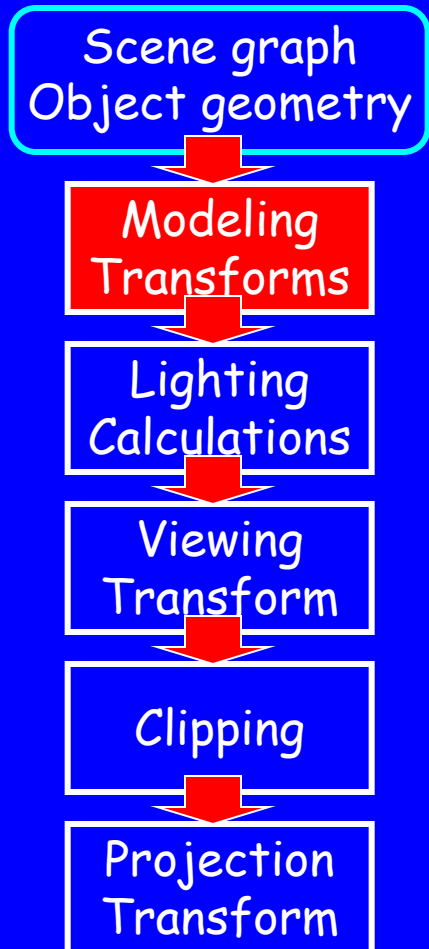
Rendering: Transformations

- So far, discussion has been in *screen space*
- But model is stored in *model space*
(a.k.a. object space or world space)
- Three sets of geometric transformations:
 - *Modeling transforms*
 - *Viewing transforms*
 - *Projection transforms*

The Rendering Pipeline: 3-D



The Rendering Pipeline: 3-D



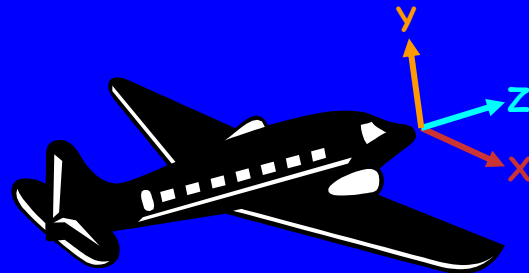
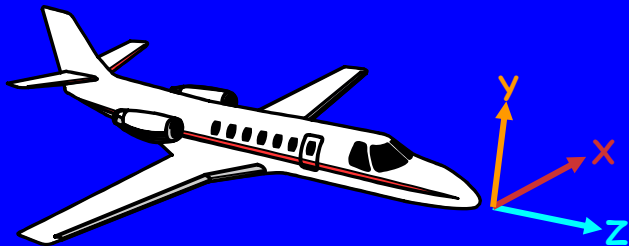
Result:

- All vertices of scene in shared 3-D "world" coordinate system

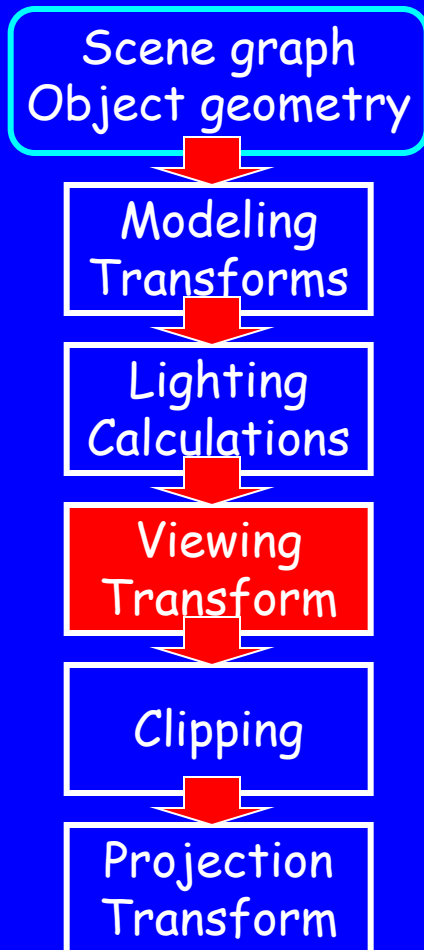


Rendering: Transformations

- Modeling transforms
 - Size, place, scale, and rotate objects and parts of the model w.r.t. each other
 - Object coordinates $\rightarrow$ world coordinates



The Rendering Pipeline: 3-D



Result:

- Scene vertices in 3-D "view" or "camera" coordinate system



Rendering: Transformations

- Viewing transform
 - Rotate & translate the world to lie directly in front of the camera
 - Typically place camera at origin
 - Typically looking down -Z axis
 - **World coordinates** $\rightarrow$ **view coordinates**

The Rendering Pipeline: 3-D

Scene graph
Object geometry

Modeling
Transforms

Lighting
Calculations

Viewing
Transform

Clipping

Projection
Transform

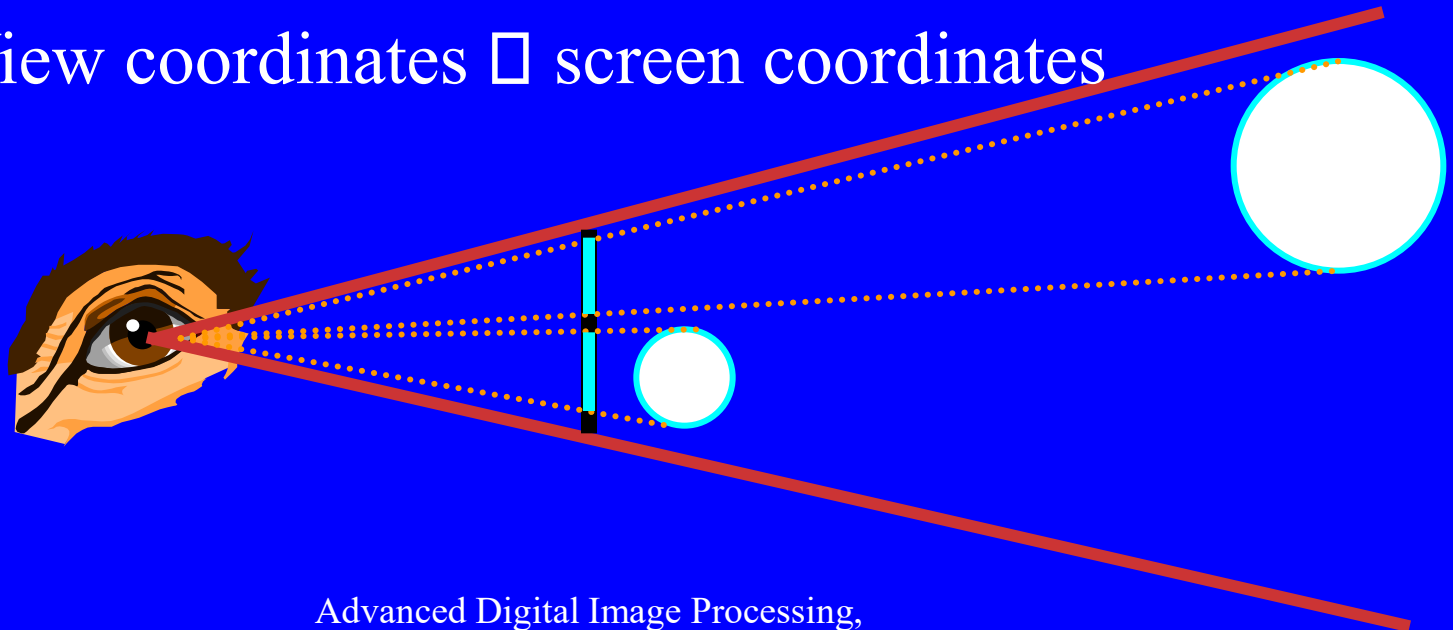
Result:

- 2-D screen coordinates of clipped vertices



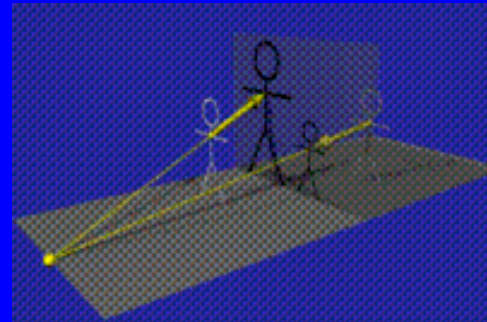
Rendering: Transformations

- Projection transform
 - Apply perspective foreshortening
 - Distant = small: the *pinhole camera* model
 - View coordinates $\mapsto$ screen coordinates

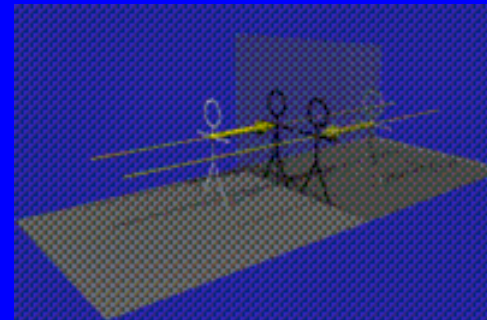


Rendering: Transformations

- Perspective Camera



- Orthographic Camera



Rendering: Transformations

- All these transformations involve shifting coordinate systems (i.e., basis sets)
- That's what matrices do...
- Represent coordinates as vectors, transforms as matrices

$$\begin{bmatrix} X' \\ Y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} X \\ Y \end{bmatrix}$$

- Multiply matrices = concatenate transforms!

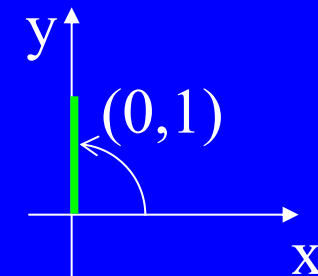
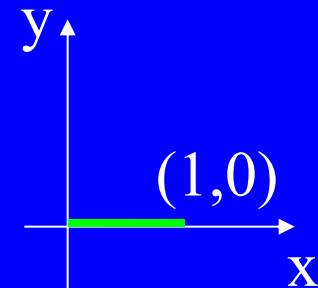
Rendering: Transformations

- Example: Rotate point $[1,0]^T$ by 90 degrees CCW (Counter-clockwise)

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos(90) & -\sin(90) \\ \sin(90) & \cos(90) \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$



Pipeline Rasterization

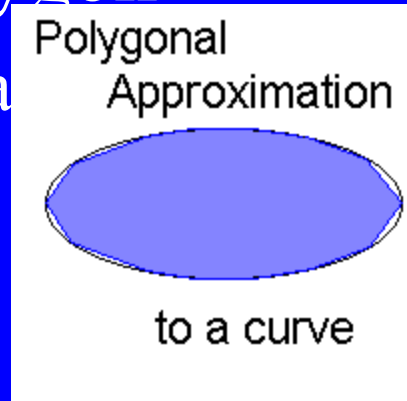
- Take a collection of 2D Polygonal Objects and draw them onto a framebuffer.
- Rasterization of Polygons is very efficient.
- All Models are eventually converted into polygons for rasterization.
- So why use other models? – Next class.

Rasterizing Polygons

- In interactive graphics, polygons rule the world
- Two main reasons:
 - Lowest common denominator for surfaces
 - Can represent any surface *with arbitrary accuracy*
 - Splines, mathematical functions, volumetric isosurfaces...
 - Mathematical simplicity lends itself to simple, regular rendering algorithms
 - Like those we're about to discuss...
 - Such algorithms embed well in hardware

Rasterizing Polygons

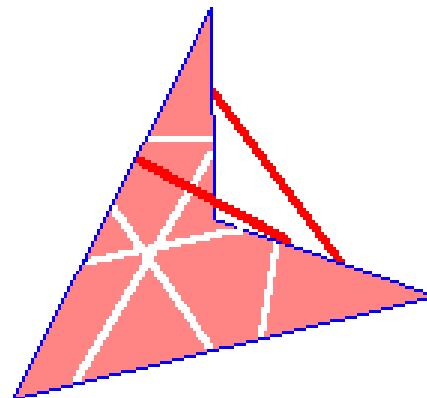
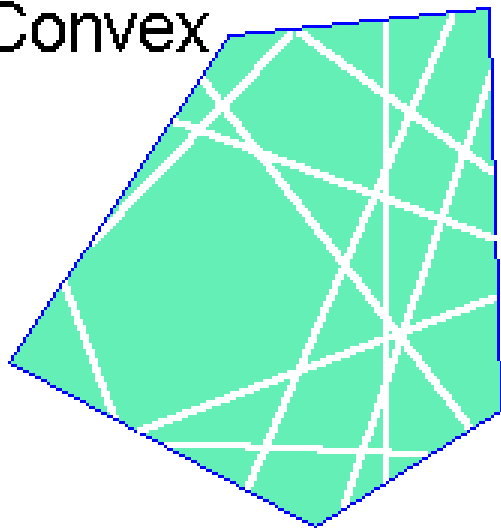
- Triangle is the *minimal unit* of a polygon
 - All polygons can be broken up into triangles
 - Convex, concave, complex
 - Triangles are guaranteed to be:
 - Planar



Convex Shapes

- A two-dimensional shape is *convex* if and only if every line segment connecting two points on the boundary is entirely contained.

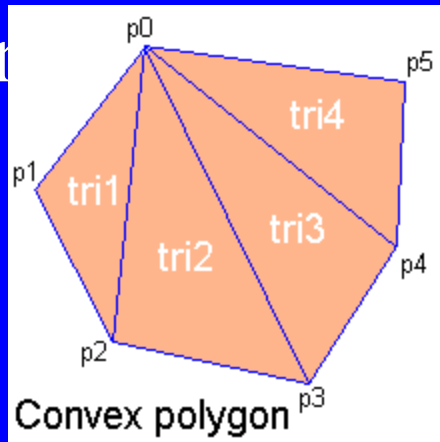
Convex



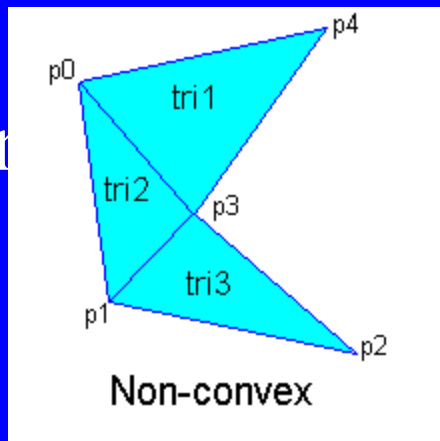
Non-convex

Triangularization

- Convex polygons are easily triangulated



- Concave polygons present a challenge

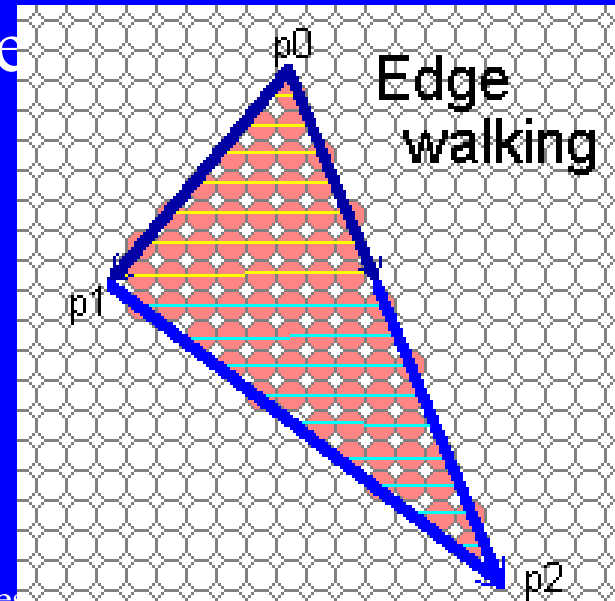


Rasterizing Triangles

- Interactive graphics hardware commonly uses *edge walking* or *edge equation* techniques for rasterizing triangles

Edge Walking

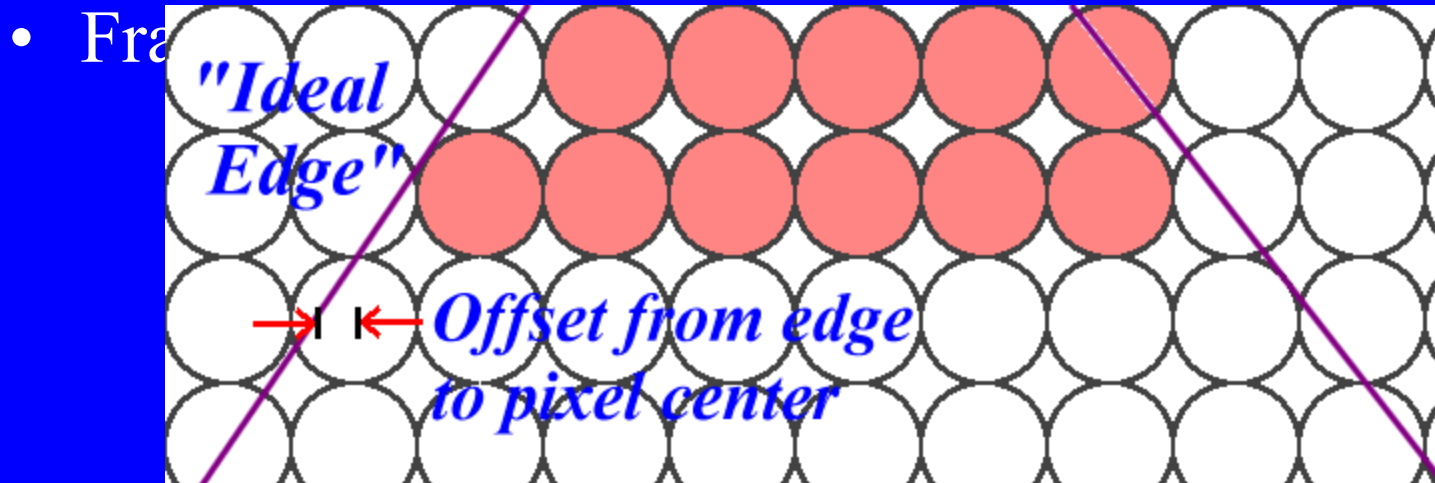
- Basic idea:
 - Draw edges vertically
 - Interpolate colors down edges
 - Fill in horizontal spans for e
 - At each scanline, interpolate edge colors across span



Edge Walking: Notes

- Order three triangle vertices in x and y
 - Find middle point in y dimension and compute if it is to the left or right of polygon. Also could be flat top or flat bottom triangle
- We know where left and right edges are.
 - Proceed from top scanline downwards
 - Fill each span
 - Until breakpoint or bottom vertex is reached
- Advantage: can be made very fast
- Disadvantages:
 - Lots of finicky special cases

Edge Walking: Disadvantages



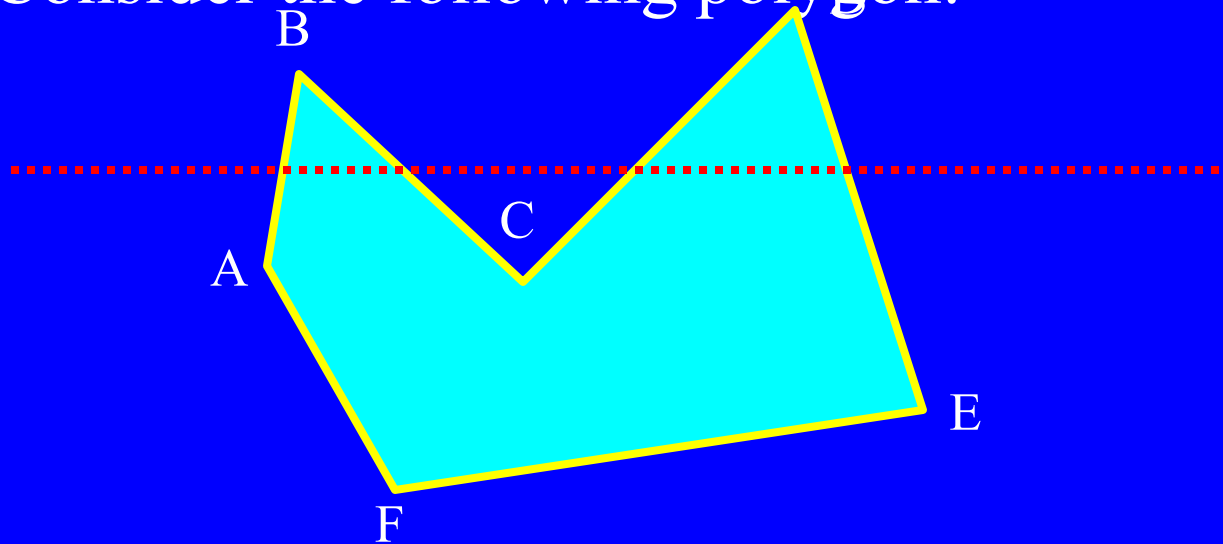
- Be careful when interpolating color values!
- Beware of gaps between adjacent edges
- Beware of duplicating shared edges

General Polygon Rasterization

- Now that we can rasterize triangles, what about general polygons?
- We'll take an edge-walking approach

General Polygon Rasterization

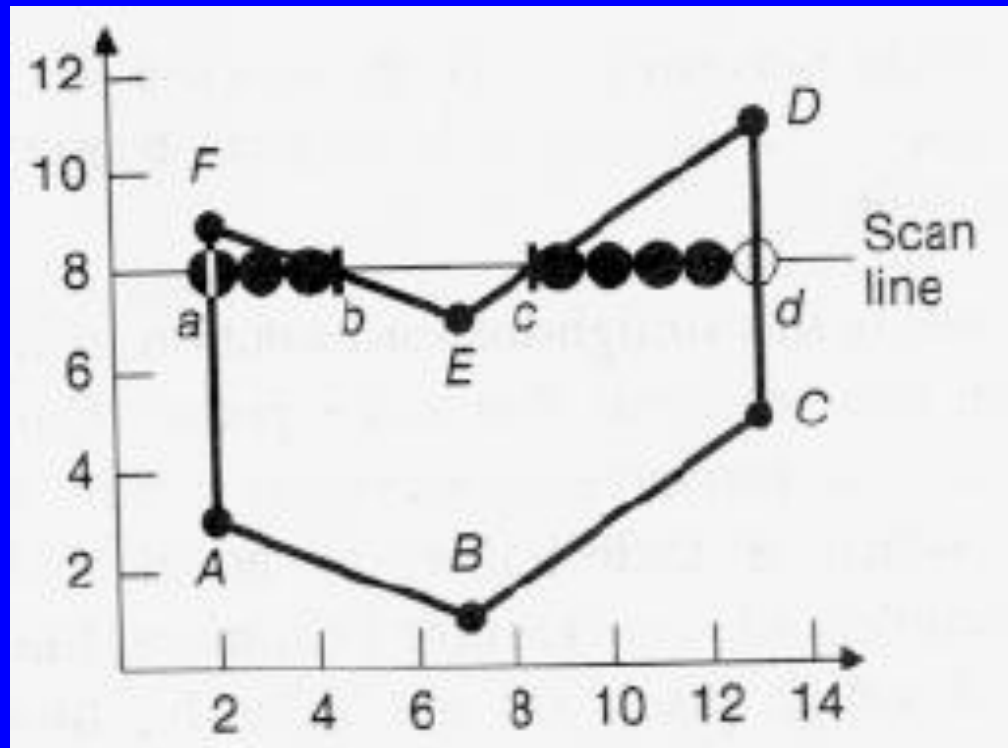
- Consider the following polygon:



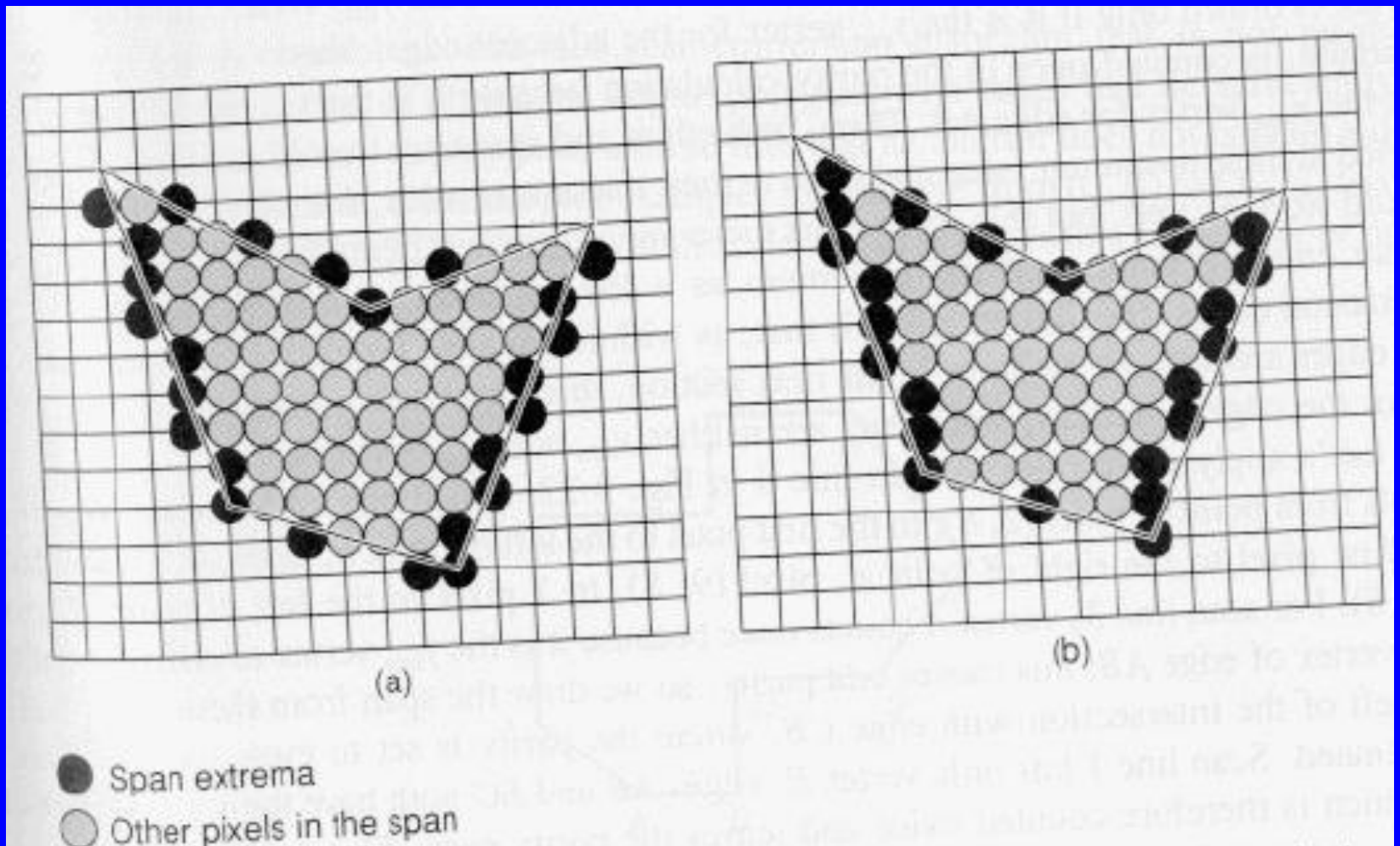
- How do we know whether a given pixel on the scanline is inside or outside the polygon?

Polygon Rasterization

- Inside-Outside Points

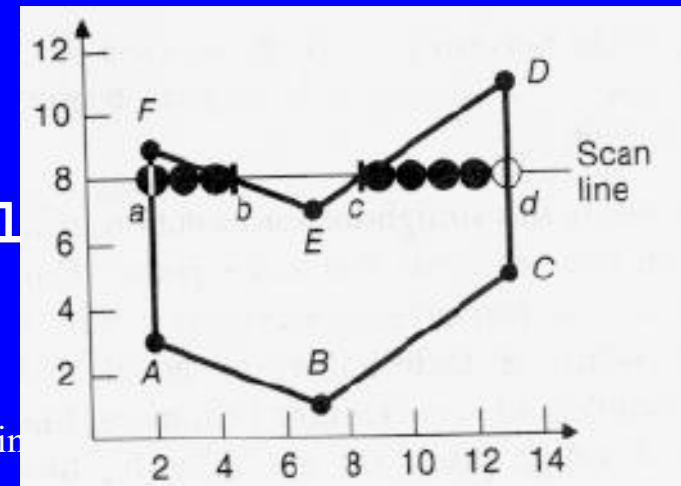


Polygon Rasterization



General Polygon Rasterization

- Basic idea: use a *parity test*
for each scanline
 edgeCnt = 0;
 for each pixel on scanline (l to r)
 if (oldpixel->newpixel crosses
edge)
 edgeCnt ++;
 // draw the pixel
 if (edgeCnt % 2)
 setPixel(pixel)



General Polygon Rasterization

- Count your vertices carefully
 - Details...

